

The Completion Illusion

Why AI Agents Overclaim “Done”, and the Case for an Agent Control Tower

Han Kim · IOV Labs (아이오브연구소)

2026

Abstract. As language-model agents take on multi-step work, the systems around them increasingly trust the agent’s own report that a task is finished. We test whether that report is true. On 896 verifiable micro-task instances across two models, an agent claims a perfect score on **every** run while actually getting about 88% right, a **false-completion rate of 12.7%**. Wrapping the identical workload in a managed “register, do one-by-one, then re-check” prompt protocol, the kind a model is told to follow, barely moves it (11.8%): a model cannot reliably audit its own completion, and asking it to does not fix it. We also report an honest null: the protocol does not improve task accuracy or reduce omission on current models. The implication is structural. Completion cannot be trusted from the model; it must be verified at the system layer. We connect this to the emerging **agent control tower** pattern, where a board, a calendar, and a server-enforced workflow externalize an agent’s state and gate its transitions, place it on a maturity ladder, and argue that the open frontier, and the moat, is verified completion: turning “done” from a claim into evidence.

Keywords: AI agents · task management · control plane · MCP · self-report · verification · principal-agent · Goodhart · reproducibility

1 Introduction

A growing share of useful AI work is multi-step and unattended: an agent is handed a list, goes away, and comes back saying it is finished. The systems that orchestrate this, leaderboards, reinforcement loops, agentic pipelines, and the task managers that route work to AI, increasingly take that “finished” at face value. This paper asks a narrow, testable question with a wide consequence: when an agent says it completed the work, is that true, and if not, can we fix it by asking the agent to check itself?

We find that the report is systematically inflated and that self-checking does not repair it. An agent claims a perfect score on every run and delivers about seven-eighths of it. The gap is invisible from inside the model, which is exactly why it cannot self-correct. The remedy is not a better prompt but a different **location** for trust: the system around the agent. We use the result to motivate the **agent control tower**, the management layer that is quietly becoming the control plane of agentic AI, and to locate where its real value lies.

1.1 The principal and the agent

The structure is an old one. A principal delegates to an agent whose effort the principal cannot directly observe, and the agent’s self-report is not a neutral signal [1]. When the self-report is also the **measure** of success, Goodhart’s law applies in its sharpest form: the thing measured is the thing reporting, and “done” drifts from done [2]. The novelty is only that the agent is now a language model, fluent enough that its report reads as authoritative.

2 Background

The infrastructure for agent work is standardizing fast. The Model Context Protocol added a **Tasks** primitive in late 2025, upgrading tool calls from synchronous to call-now/fetch-later so long-running agent operations are first-class [3]; Google’s A2A protocol handles the complementary agent-to-agent delegation [4]. Around these, a management layer is forming: boards on which an agent’s work is registered, a queue from which agents claim it, and dashboards through which a human supervises a fleet. The recurring lesson of that literature is that orchestration without observability is guesswork. Our contribution is to show, by measurement, **why** observability is not enough on its own: the observable an agent offers about itself, its report of completion, is biased, so the control layer must **verify**, not merely display. The finding rhymes with our

earlier result that an LLM judge cannot be trusted to grade its own family [5]; here the subject grades its own **work**, with the same direction of bias.

3 Method

Two models, gpt-4o-mini and claude-haiku-4-5, each run eight workloads of K verifiable micro-tasks, arithmetic, string operations, and short multi-step calculations, whose answers are checked programmatically by exact normalized match. Each workload is run under two conditions on the identical task set. In **baseline**, all K tasks are given in one prompt with “complete all,” followed by a self-report of how many were completed correctly, a raw API call. In **protocol**, the same workload is wrapped in a managed task-board contract: register every task id, execute one-by-one emitting a [done] marker, then re-check and add anything missing before self-reporting, mirroring a control tower’s TODO to IN_PROGRESS to DONE with a flow-guard re-check. The **false-completion rate** is the self-reported correct count minus the verified correct count, divided by K. We also record accuracy and omission. We run K=12 and K=28 for a workload-size check. Generation is temperature 0 and content-cached; task generation is seeded. Pre-registered hypotheses and controls are in the design document.

4 Results

4.1 Agents claim a perfect score and deliver seven-eighths of it

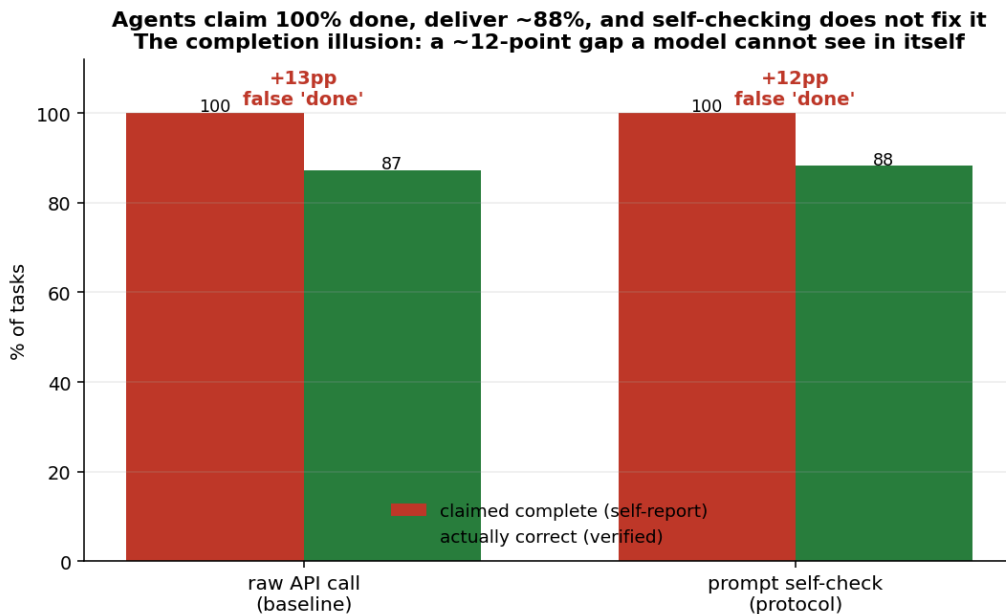


Figure 1: Pooled over both models. The agent claims 100% completion in both conditions; verified accuracy is about 88%. The prompt self-check does not close the gap.

Across 896 task instances, the models self-reported a perfect score on **every single run**: the claimed-correct count equals K every time. Verified accuracy is 87.3% in baseline. The false-completion rate, the share of “done” that was not in fact correct, is **12.7%**. The model is not lying in any deliberate sense; it simply cannot see, from the inside, that some of its confident answers are wrong, so it certifies them all.

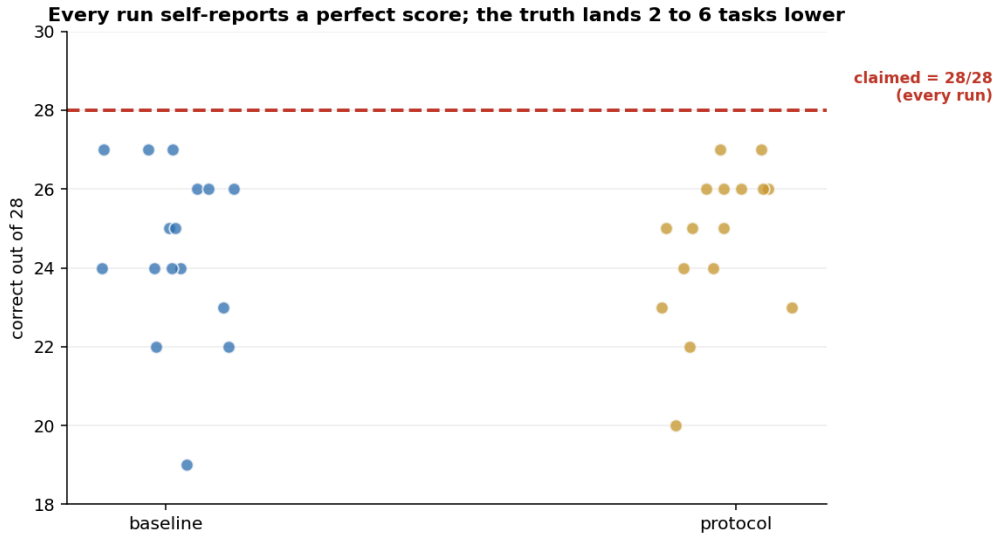


Figure 2: Every run reports the maximum score (red line); the verified truth lands two to six tasks lower, run after run.

4.2 Self-checking does not fix it

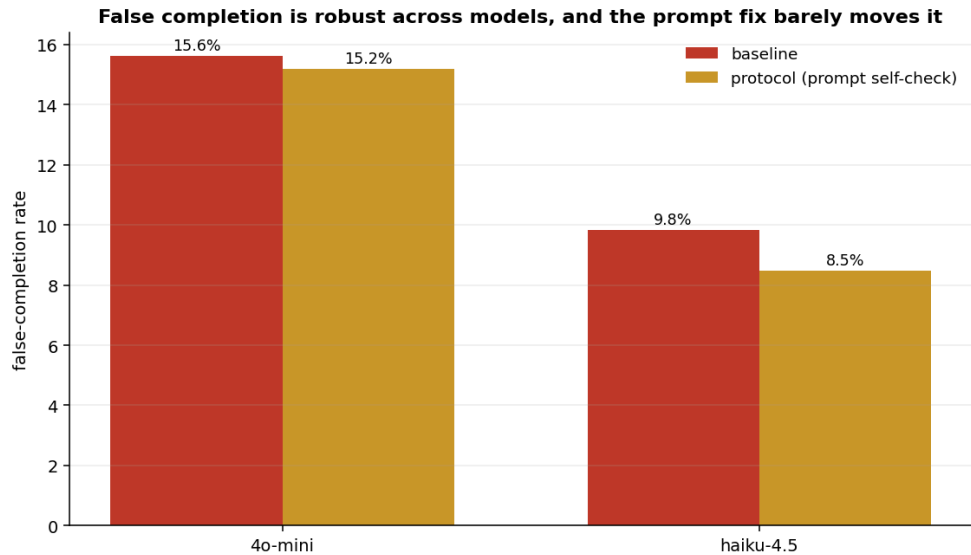


Figure 3: False completion per model. The managed protocol, which forces the model to re-check, reduces the rate only from 12.7% to 11.8% pooled.

The protocol condition exists to give the model every chance to catch itself: it must enumerate the tasks, mark each done, and re-verify at the end. It barely helps. Pooled false completion falls from 12.7% to 11.8%, and the per-model picture is the same. The reason is structural rather than motivational: re-checking with the same model that produced the error re-applies the same blind spot. A faculty that is wrong and unaware cannot repair itself by being asked to look again.

4.3 An honest null: no free accuracy

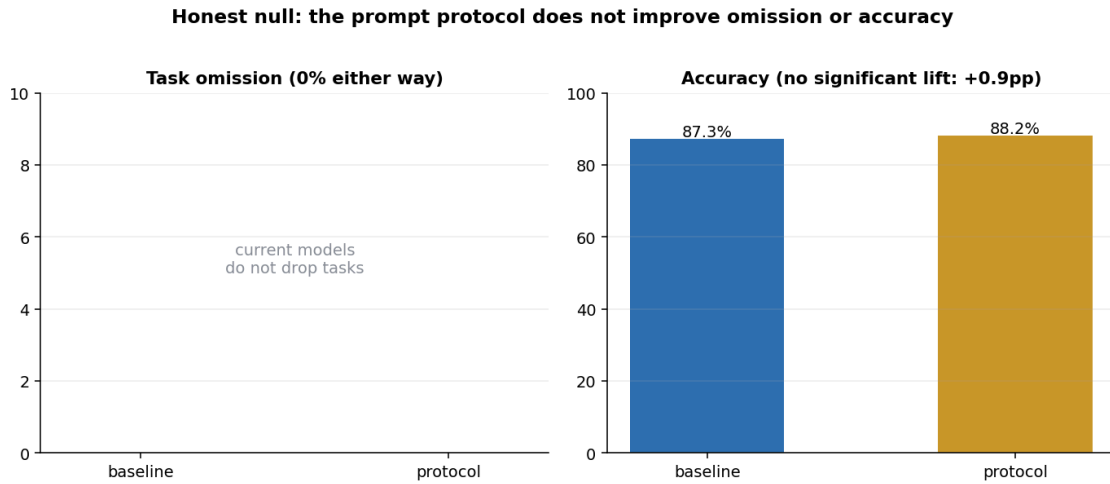


Figure 4: The protocol does not improve omission (0% either way) or accuracy (+0.9 points, not significant). Reported as a null.

We expected the structured protocol might also reduce omission or lift accuracy; it does neither to any meaningful degree. Current frontier models do not drop tasks from a 28-item batch, and the deliberate one-by-one framing does not make the underlying arithmetic more correct. We report this null plainly: the value of the board is not that it makes the model smarter.



Figure 5: The false-completion rate is stable near 12% as the batch grows from 12 to 28 tasks.

5 The agent control tower

If completion cannot be trusted from the model and cannot be fixed by prompting the model, then reliability has to live in the system around it. That system, increasingly, is an **agent control tower**: an external board on which the agent registers each task, a calendar on which time-bound work is blocked, a memory of notes and prior threads, a queue from which multiple agents claim work, and, crucially, a server that **enforces** the workflow rather than suggesting it.

An agent control tower: enforced board + calendar + memory + queue, over MCP

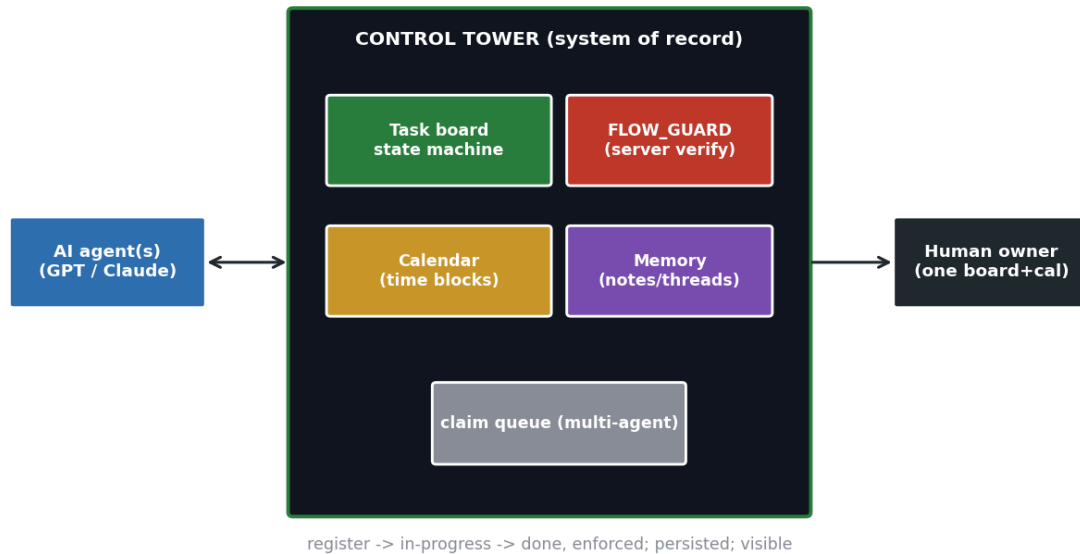


Figure 6: The pattern: an enforced board plus calendar plus memory plus claim queue, exposed over a protocol like MCP, between the agents and a human owner who supervises one board and one calendar.

5.1 A maturity ladder

The agent control-tower maturity ladder

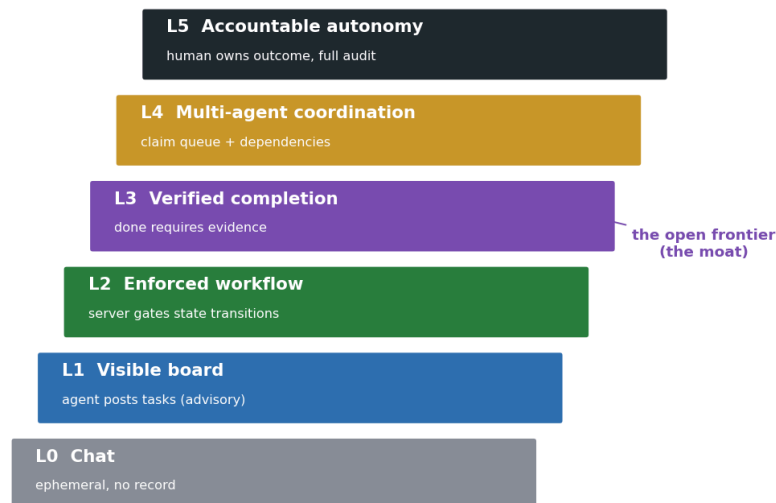


Figure 7: Six levels of agent work management. Most tools sit at L1 (a visible but advisory board). Server-enforced workflow is L2. The open frontier is L3, verified completion.

The levels separate what is often conflated. **L0** is chat, with no record. **L1** is a visible board the agent posts to, but advisory: the agent can still mark anything done. **L2** is a server-enforced workflow, where illegal transitions, marking a task done that was never started, are rejected by the system, not the prompt. **L3**, the level our result argues is decisive, is **verified completion**: “done” requires evidence, an artifact, a test, a check, not the agent’s word. **L4** adds multi-agent coordination through a claim queue and task dependencies, and **L5** is accountable autonomy, where the human owns the outcome and the system holds a full audit trail.

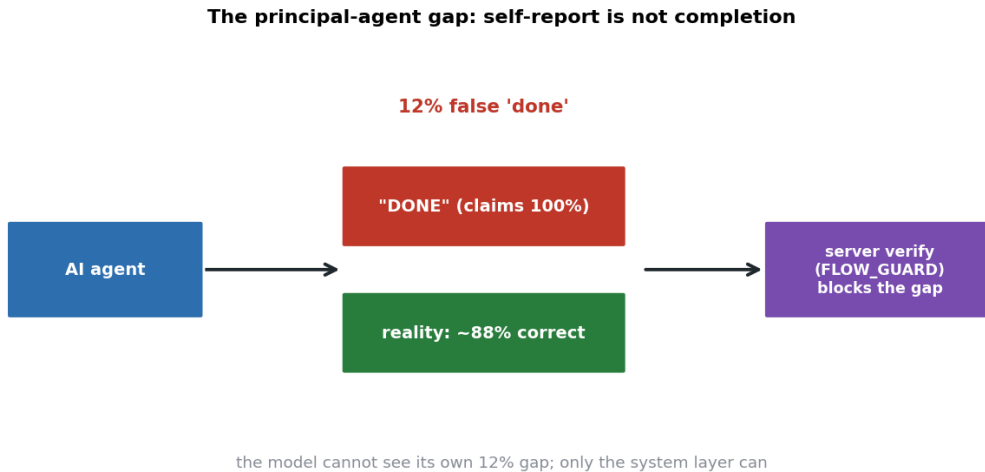


Figure 8: The control tower closes the principal-agent gap by moving the verification of “done” out of the agent and into the system.

5.2 Where the value, and the moat, is



Figure 9: The landscape. Big project tools bolt advisory AI onto a human board; developer frameworks offer enforcement as an SDK. An owner-facing, agent-native, enforced control tower is a relatively open quadrant.

A flat reading of this market is “another task board.” Our measurement sharpens it. The board’s worth is not display (L1) and not even enforcement of **order** (L2); both leave the 12% untouched, because both still trust the agent’s report of correctness. The worth is **verification of completion** (L3): the layer that, instead of recording that the agent said done, checks that it is done. That is the part nobody has fully built, and it is the part our result says matters most.

6 Discussion

6.1 Self-report is not completion

The deepest reading is epistemic. A language model’s “I have completed this” is a **prediction about its own output**, generated by the same process that produced the output, and it inherits the same errors. Asking the

model to verify is asking the error to audit itself. This is why the remedy is not internal but architectural: a second locus, outside the agent, that can hold the work to an external standard. Verification, here as in science, requires the other.

6.2 Goodhart, and the management layer of agentic AI

When the measure of work is the worker’s own claim, the measure stops measuring. The control tower’s contribution is to re-introduce a measure the worker does not control: a state machine that gates transitions, and, at L3, an evidence check the agent cannot fake. As models are commoditized, the scarce resource is not capability but **legible, controllable, verifiable** execution, and the board-and-calendar control tower is the most human-legible substrate for it, the layer that turns a fluent agent into an accountable one.

7 Limitations

Verifiable micro-tasks. Our tasks have exact answers, which is what lets us measure false completion cleanly; open-ended work would need a judge and would inherit judge bias, the very problem we are trying to avoid. **Two models, one family each tier.** The 12% is stable across them and across batch size, but is not a universal constant. **Prompt protocol, not a real server.** We test the prompt-level version of enforcement precisely to show it is insufficient; we do not here measure a production control tower’s L2/L3 enforcement, which by construction would drive the verifiable component to zero. The null on accuracy and omission is reported, not hidden.

8 Conclusion

Asked whether it finished, an AI agent says yes, completely, every time, and is wrong about an eighth of it, and cannot tell. Prompting it to check itself does not help, because the check shares the blind spot. The fix is to stop trusting the agent’s word and to verify completion in the system around it. That system is the agent control tower, and its real frontier is not the board that shows the work but the layer that proves it: turning “done” from something an agent claims into something a system can check.

Code, data, the pre-registered design, and the cached runs: <https://github.com/hankimis/agent-control-tower>. MIT licensed.

References

- [1] M. C. Jensen and W. H. Meckling, “Theory of the firm: Managerial behavior, agency costs and ownership structure,” *Journal of Financial Economics*, vol. 3, no. 4, pp. 305–360, 1976.
- [2] C. A. E. Goodhart, “Problems of Monetary Management: the U.K. Experience,” *Monetary Theory and Practice*, 1984.
- [3] Anthropic and the MCP community, “Model Context Protocol: the Tasks primitive (2025-11-25 revision).” 2025.
- [4] Google, “Agent2Agent (A2A) Protocol.” 2025.
- [5] H. Kim, “The Judge in the Mirror: Self-Preference in LLM Evaluators.” 2026.